

EXPERIMENT-9

Write a program for A* search algorithm using python code

Aim : To implement the A* search algorithm using Python.

Procedure:

Define the graph with edge costs.

Define the heuristic values for each node.

Start from the source node.

Select the node with the lowest $f(n) = g(n) + h(n)$.

Continue until the goal node is reached.

Display the shortest path.

PYTHON CODE:

```
import heapq
```

```
graph = {
```

```
    'A': [('B', 1), ('C', 3)],
```

```
    'B': [('D', 3), ('E', 6)],
```

```
    'C': [('F', 2)],
```

```
    'D': [('G', 3)],
```

```
    'E': [('G', 2)],
```

```
    'F': [('G', 1)],
```

```
    'G': []
```

```
}
```

```
heuristic = {
```

```
    'A': 6,
```

```
    'B': 4,
```

```
    'C': 3,
```

```
    'D': 2,
```

```
    'E': 1,
```

```
    'F': 1,
```

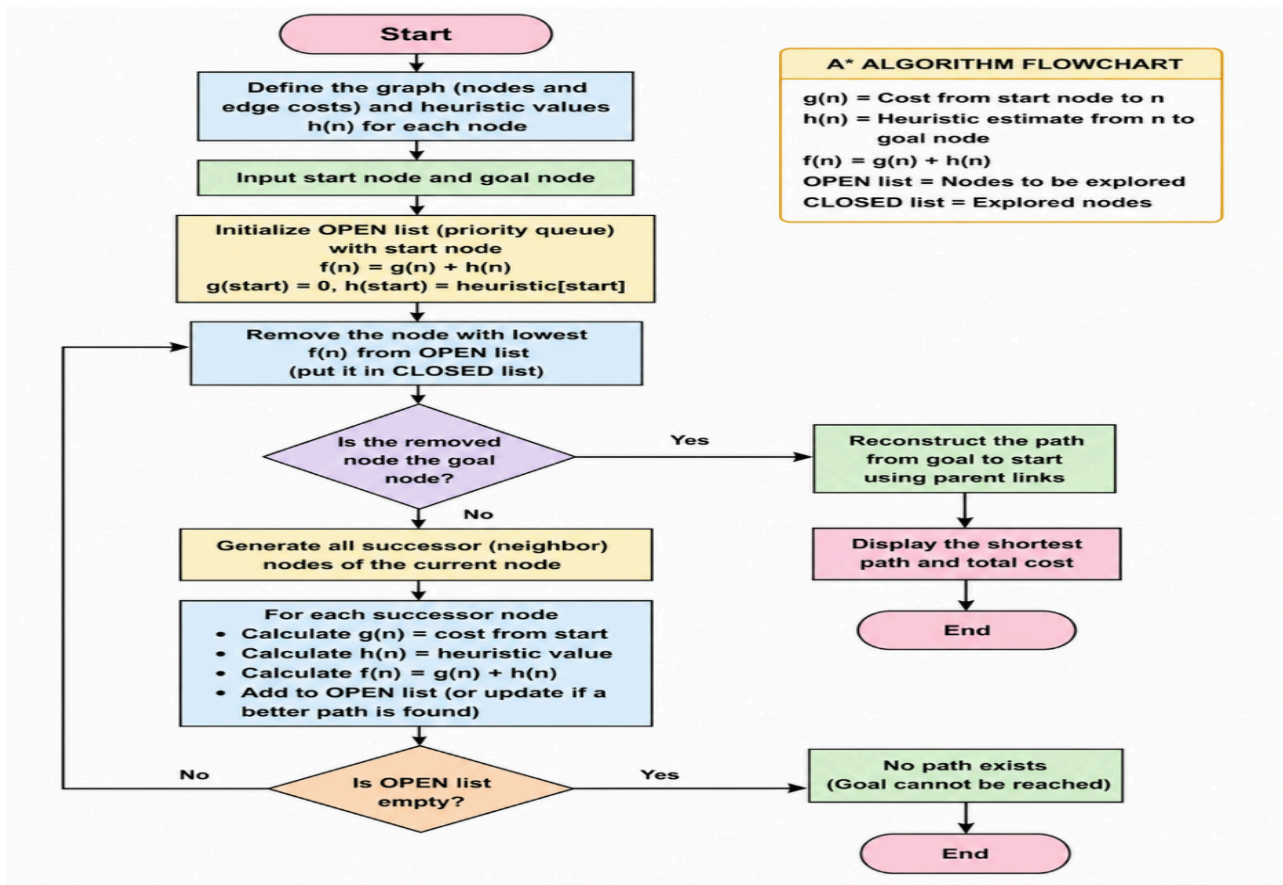
```

    'G': 0
}

def astar(start, goal):
    pq = [(heuristic[start], 0, start, [start])]
    visited = set()
    while pq:
        f, g, node, path = heapq.heappop(pq)
        if node == goal:
            return path, g
        if node in visited:
            continue
        visited.add(node)
        for neighbour, cost in graph[node]:
            if neighbour not in visited:
                new_g = g + cost
                new_f = new_g + heuristic[neighbour]
                heapq.heappush(pq, (new_f, new_g, neighbour, path + [neighbour]))
    path, cost = astar('A', 'G')
    print("Shortest Path:")
    print(" -> ".join(path))
    print("Total Cost =", cost)

```

FLOWCHART:



Output:

```

graph = {
  'A': [( 'B', 1), ( 'C', 3)],
  'B': [( 'D', 3), ( 'E', 1)],
  'C': [( 'F', 5)],
  'D': [],
  'E': [( 'G', 2)],
  'F': [( 'G', 1)],
  'G': []
}

h = {'A': 5, 'B': 4, 'C': 3, 'D': 3, 'E': 2, 'F': 1, 'G': 0}

open_list = [( 'A', 0)]
visited = set()

while open_list:
    open_list.sort(key=lambda x: x[1] + h[x[0]])
    node, cost = open_list.pop(0)

    if node == 'G':
        print("Goal reached with cost:", cost)
        break

    if node not in visited:
        visited.add(node)
        for neighbor, weight in graph[node]:
            open_list.append((neighbor, cost + weight))
  
```

Goal reached with cost: 4

=== Code Execution Successful ===

Result: Hence, A* (A-Star) search algorithm has solved

